ML LAB 4

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import roc_auc_score

import lightgbm as lgb
import xgboost as xgb
```

```
train = pd.read_csv('train.csv')
test = pd.read_csv('test.csv')
sample_submission = pd.read_csv('sample_submission.csv')

print(train.shape, test.shape)
```

```
(750000, 18) (250000, 17)
```

```
TARGET = 'y'
X = train.drop(columns=[TARGET])
y = train[TARGET]
```

```
cat_cols = X.select_dtypes(include=['object']).columns.tolist()
num_cols = X.select_dtypes(exclude=['object']).columns.tolist()

print("Categorical:", cat_cols)
print("Numerical:", num_cols)
```

```
Categorical: ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']
Numerical: ['id', 'age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous']
```

```
numeric_transformer = 'passthrough'
categorical_transformer = OneHotEncoder(handle_unknown='ignore')

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, num_cols),
        ('cat', categorical_transformer, cat_cols),
    ])
```

```
lgb_model = lgb.LGBMClassifier(
    objective='binary',
    metric='auc',
    learning_rate=0.05,
    n_estimators=800,
    num_leaves=31,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    force_row_wise=True,
    verbosity=-1
)


model = Pipeline([
    ('preprocessor', preprocessor),
    ('model', lgb_model)
])
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
auc_scores = []

for fold, (train_idx, val_idx) in enumerate(skf.split(X, y), 1):
    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
```

```
    y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]

    model.fit(X_train, y_train)
    y_val_pred = model.predict_proba(X_val)[:,1]
    score = roc_auc_score(y_val, y_val_pred)
    auc_scores.append(score)
    print(f"Fold {fold} AUC: {score:.4f}")

print(f"\nMean CV AUC: {np.mean(auc_scores):.4f}")
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, bu
  warnings.warn(
Fold 1 AUC: 0.9683
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, bu
  warnings.warn(
Fold 2 AUC: 0.9673
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, bu
  warnings.warn(
Fold 3 AUC: 0.9671
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, bu
  warnings.warn(
Fold 4 AUC: 0.9683
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, bu
  warnings.warn(
Fold 5 AUC: 0.9676

Mean CV AUC: 0.9677
```

```
 model.fit(X, y)
y_test_pred = model.predict_proba(test)[:,1]

sample_submission[TARGET] = y_test_pred
sample_submission.to_csv('submission.csv', index=False)
print("Submission file saved!")
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, bu
  warnings.warn(
Submission file saved!
```